

[Click To Skip To Main Content](#)

- [Home](#)
- [Über Mich](#)
- [Leistungen](#)
- [Portfolio](#)
- [Kontakt](#)
- [BLOG](#)

Was ist ein Headless CMS? – Und warum Sanity mein Lieblings-Tool ist



Ivo S. 05.04.2025 Lesezeit: 11:02

Hallo und herzlich willkommen zurück auf meinem Blog! 🙌

Heute tauchen wir tief in ein Thema ein, das aus der Welt der Webentwicklung und Content-Verwaltung nicht mehr wegzudenken ist: **Headless Content Management Systeme (CMS)**.

Vielleicht hast du schon davon gehört, vielleicht fragst du dich, was dieser „kopflose“ Ansatz überhaupt soll. Keine Sorge, genau das klären wir heute!

Wir schauen uns an, was ein Headless CMS von einem klassischen System wie WordPress (in seiner traditionellen Form) unterscheidet und warum gerade Lösungen wie [Sanity.io](#) für uns Entwickler so unglaublich spannend sind.

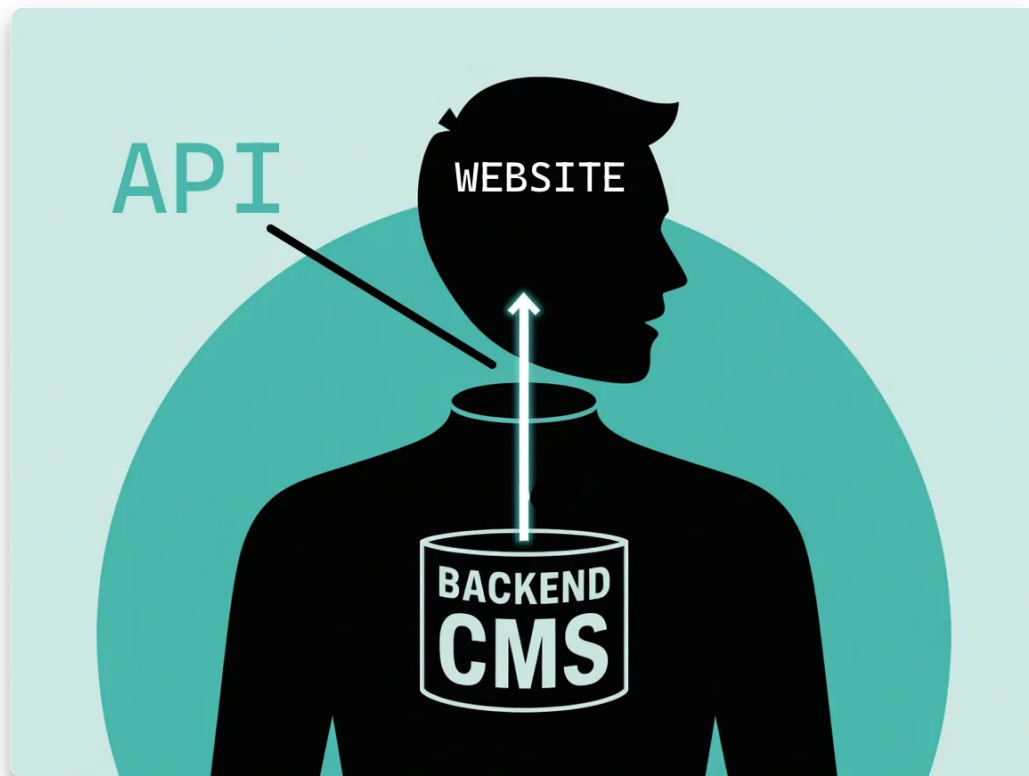
Das Beste daran: Ich zeige dir nicht nur graue Theorie, sondern nehme dich mit hinter die Kulissen meines eigenen Blog-Projekts hier auf [blackforest-webcraft.de](#). Du erfährst, warum ich mich für Sanity entschieden habe und wie ich es ganz konkret nutze, um die Inhalte zu verwalten, die du gerade liest.

Also, schnell dich an, es wird technisch, aber praxisnah und (hoffentlich) spannend für dich!

Inhaltsverzeichnis 📄

1. [Der Kopf muss ab? Klassisches vs. Headless CMS](#)
2. [Was ist Sanity CMS? Das Backend neu gedacht](#)
3. [Vorteile eines Headless CMS am Beispiel von Sanity](#)
4. [Use Case: Mein eigener Blog \(blackforest-webcraft.de\)](#)
5. [Fazit: Sanity – Flexibel, Modern, Entwicklerfreundlich](#)

1. Der Kopf muss ab? Klassisches vs. Headless CMS



Stell dir ein traditionelles CMS wie WordPress, Joomla oder Drupal vor. Diese Systeme sind oft „monolithisch“. Das bedeutet, das Backend (wo du Inhalte erstellst und verwaltest) und das Frontend (das, was deine Besucher sehen – die eigentliche Website mit Design und Layout) sind fest miteinander verbunden. Du wählst ein Theme, passt es vielleicht etwas an, schreibst deine Beiträge im Backend, und das CMS rendert die fertige HTML-Seite für den Browser. Das funktioniert für viele Anwendungsfälle super und hat sich über Jahre bewährt.

Aber was, wenn du mehr willst? Was, wenn du nicht an ein bestimmtes Theme-System gebunden sein möchtest? Was, wenn du deine Inhalte nicht nur auf einer Webseite, sondern vielleicht auch in einer mobilen App, auf einer Smartwatch oder einem digitalen Infoscreen anzeigen willst? Hier kommen **Headless CMS** ins Spiel.

Mit Headless-Systemen gibt es in der Entwicklung quasi keine Grenzen – genau wie in meiner eigenen Philosophie im Webentwicklung-Service. Deshalb arbeite ich nicht mit Baukastensystemen, sondern setze bewusst auf individuellen Custom Code. Aber zurück zum Thema:

Der Name "Headless" (kopfflos) ist eigentlich ganz treffend: Man trennt den „**Kopf**“ (das **Frontend**, die Präsentationsebene) vom „**Körper**“ (dem **Backend**, der Content-Verwaltung und -Speicherung). Übrig bleibt ein reines Backend-System, das sich ausschließlich auf die Organisation und Bereitstellung von Inhalten konzentriert.

Wie kommen die Inhalte dann zum Nutzer? Über **APIs** (Application Programming Interfaces). Das Backend stellt die Inhalte über eine standardisierte Schnittstelle zur Verfügung, und du als Entwickler kannst diese Daten mit jeder beliebigen Technologie abrufen und darstellen. Ob du eine Website mit React, Vue, Svelte, Next.js, Astro baust oder – wie in meinem Fall – mit klassischem HTML, CSS und JavaScript, bleibt ganz dir überlassen. Du hast die volle Kontrolle über das Frontend.

Warum ist das für Entwickler spannend?

- **Technologiefreiheit:** Du bist nicht mehr an die Template-Sprache oder die Architektur des CMS gebunden. Du nutzt die Tools und Frameworks, die du liebst und die am besten zum Projekt passen.
- **Flexibilität:** Inhalte können für verschiedenste Kanäle (Website, App, etc.) wiederverwendet werden – "Create Once, Publish Everywhere".
- **Performance & Sicherheit:** Oftmals führen Headless-Architekturen zu schnelleren Ladezeiten (z. B. durch Static Site Generation) und einer verbesserten Sicherheit, da das Backend vom öffentlich zugänglichen Frontend getrennt ist.
- **Moderne Workflows:** Headless CMS passen oft besser zu modernen Entwicklungspraktiken wie Git-basierten Workflows, CI/CD und JAMstack-Architekturen.

Genau hier setzt [Sanity.io](#) an, ein besonders flexibles und entwicklerfreundliches Headless CMS, das ich für meinen Blog gewählt habe. Lass uns das mal genauer anschauen.

2. Was ist Sanity CMS? Das Backend neu gedacht

[Sanity.io](#) ist also ein Vertreter der Headless-CMS-Gattung. Aber was macht es besonders? Hier sind die Kernkonzepte:

Der gute CMS Kellner

- **Headless durch und durch:** Wie besprochen, Sanity ist *nur* das **Backend**. Es kümmert sich um die Speicherung, Strukturierung und Bereitstellung deiner Inhalte. Dein Frontend baust du komplett separat. Diese Trennung ist fundamental.
- **Inhalte über APIs:** Sanity stellt deine Inhalte über mächtige APIs bereit. Du hast die Wahl zwischen der hauseigenen Abfragesprache **GROQ** (Graph-Relational Object Queries), die unglaublich flexibel ist und es dir erlaubt, Daten genau in der Form abzurufen, wie du sie brauchst – inklusive Verknüpfungen und Transformationen direkt in der Abfrage. Alternativ steht auch eine klassische **REST-API** zur Verfügung.
- Stell dir die API wie einen Kellner vor: Du (dein Frontend) sagst ihm genau, welche Daten (Gerichte) du brauchst, und er liefert sie dir aus der Küche (dem Sanity Content Lake). **BILD VON KELLNER!!!**

Sanity Schema Beispiel

- **Strukturierte Inhalte durch Schemas:** Das ist einer der mächtigsten Aspekte von Sanity! Du definierst die Struktur deiner Inhalte komplett selbst. Anstatt vorgegebener Felder wie "Titel" und "Text" (obwohl du die natürlich auch haben kannst), definierst du sogenannte **Schemas** in JavaScript oder TypeScript. Du legst fest, welche Inhaltstypen es gibt (z. B. "Blogbeitrag", "Projekt", "Autor") und welche Felder jeder Typ hat (z. B. Titel, Slug für die URL, Einleitungstext, Haupttext, Bildergalerie, Keywords, Veröffentlichungsdatum). Du bestimmst Datentypen (Text, Zahl, Datum, Bild, Verweis auf andere Inhalte etc.) und kannst sogar Validierungsregeln festlegen. Das Ergebnis: Deine Inhalte sind perfekt auf deine Bedürfnisse zugeschnitten und nicht umgekehrt. Stell dir das wie einen individuellen Baukasten vor, den du selbst entwirfst. ✂
- **Sanity Studio: Dein maßgeschneidertes Cockpit:** Wo verwaltest du nun diese Inhalte? Im **Sanity Studio**. Das ist eine Open-Source-Anwendung (gebaut mit React, aber keine Sorge, du musst kein React können, um es zu nutzen oder anzupassen!), die du selbst hosten oder von Sanity hosten lassen kannst. Das Studio ist dein Redaktions-Backend. Das Coole daran: Weil es auf deinen selbst definierten Schemas basiert, passt sich das Studio genau an deine Inhaltsstrukturen an. Du kannst das Studio weitreichend konfigurieren und anpassen, um den Redakteuren (oder dir selbst) den bestmöglichen Workflow zu bieten. Es ist modern, schnell und bietet Features wie Echtzeit-Zusammenarbeit und Live-Vorschauen.

Zusammengefasst: Sanity gibt dir die Werkzeuge, um deine Inhalte auf eine hochgradig strukturierte und flexible Weise zu verwalten und sie über leistungsstarke APIs für jedes beliebige Frontend bereitzustellen.

3. Vorteile eines Headless CMS am Beispiel von Sanity

Okay, die Konzepte klingen gut. Aber was sind die konkreten Vorteile, wenn du Sanity für dein Projekt wählst? Lass uns das mal aufdröseln:

- **Absolute Frontend-Freiheit:** Das ist der offensichtlichste Vorteil. Da Sanity nur die Daten liefert, bist du völlig frei in der Wahl deiner Frontend-Technologie.
 - Du willst eine blitzschnelle statische Seite mit **Next.js** oder **Astro**? Kein Problem.
 - Du bevorzugst eine Single Page Application mit **React**, **Vue** oder **Svelte**? Go for it!
 - Du möchtest, wie ich, auf Frameworks verzichten und lieber mit **reinem HTML, CSS und JavaScript** arbeiten, um maximale Kontrolle zu haben oder die Grundlagen zu festigen? Auch das ist problemlos möglich!
 - Du brauchst die Daten in einer **nativen mobilen App** (iOS/Android)? Hol sie dir einfach per API. Sanity schreibt dir nichts vor. Du entscheidest, was für dein Projekt und deine Fähigkeiten am besten passt.
- **Multichannel-Content leicht gemacht:** Da der Inhalt von der Präsentation getrennt ist, kannst du denselben Inhalt an vielen Orten gleichzeitig ausspielen. Schreibe einen Blogbeitrag in Sanity und zeige ihn auf deiner Website, in einem Newsletter und vielleicht sogar auszugsweise in einer App an. Definiere Produktinformationen einmal und nutze sie im Webshop, in einer internen App für den Vertrieb und auf digitalen Displays im Laden. Das spart Zeit und sorgt für Konsistenz.
- **Individuelle Schemas für perfekte Inhaltsmodellierung:** Das habe ich schon erwähnt, aber es ist so wichtig, dass es eine eigene Erwähnung verdient. Du bist nicht mehr gezwungen, deine Inhalte in vorgefertigte Schablonen zu pressen. Du definierst die Struktur exakt so, wie dein Projekt sie erfordert.
 - Brauchst du spezielle Felder für SEO-Metadaten? Füge sie hinzu.
 - Willst du komplexe Beziehungen zwischen verschiedenen Inhaltstypen abbilden (z. B. Autoren zu Artikeln, Produkte zu Kategorien)? Sanity macht das einfach.
 - Benötigst du einen Baukasten für Landing Pages, bei dem Redakteure verschiedene Module (Textblöcke, Bilder, Videos, CTAs) flexibel anordnen können? Mit Sanity's Array- und Object-Typen ist das elegant lösbar ("Portable Text" ist hier ein Stichwort – extrem mächtig!). Diese maßgeschneiderte Struktur macht nicht nur die Content-Pflege logischer, sondern auch die Entwicklung des Frontends einfacher, weil die Daten genau so ankommen, wie du sie erwartest.
- **Schnell, sicher und skalierbar:**
 - **Schnell:** Headless-Architekturen ermöglichen oft den Einsatz von Static Site Generators (SSGs) oder serverseitigem Rendering (SSR) mit Frameworks wie Next.js, was zu extrem schnellen Ladezeiten führt. Die APIs von Sanity sind ebenfalls auf Performance optimiert.
 - **Sicher:** Die Angriffsfläche wird reduziert. Dein Frontend kann eine statische Seite sein oder über eine Serverless-Funktion laufen, während das eigentliche CMS (Sanity) sicher in der Cloud liegt und nicht direkt dem öffentlichen Internet ausgesetzt ist (außer über die gesicherten APIs).
 - **Skalierbar:** Sanity ist als Cloud-native Plattform konzipiert. Der "Content Lake" (die Datenbank) ist darauf ausgelegt, mit deinen Anforderungen zu wachsen, ohne dass du dich um Serverwartung oder Skalierungsgengpässe kümmern musst.
- **Moderne Redaktions-Experience mit Live-Preview & Echtzeit-Editing:** Das Sanity Studio ist nicht nur für Entwickler toll, sondern auch für Redakteure. Es bietet eine saubere, anpassbare Oberfläche. Besonders hervorzuheben sind:
 - **Echtzeit-Zusammenarbeit:** Ähnlich wie in Google Docs können mehrere Bearbeiter gleichzeitig an Inhalten arbeiten und sehen die Änderungen der anderen live.
 - **Live-Preview:** Du kannst das Studio so konfigurieren, dass es eine Live-Vorschau deines Frontends direkt neben dem Bearbeitungsformular anzeigt.

Änderungen im Studio werden (fast) in Echtzeit in der Vorschau sichtbar. Das ist Gold wert für Redakteure, die sofort sehen wollen, wie ihre Inhalte später aussehen werden.

Diese Kombination aus Flexibilität, Kontrolle und moderner Technologie macht Sanity zu einer attraktiven Wahl für eine Vielzahl von Projekten – von persönlichen Blogs bis hin zu großen Unternehmens-Websites und komplexen Anwendungen.

4. Use Case: Mein eigener Blog (blackforest-webcraft.de)

Jetzt wird's konkret! Wie nutze ich Sanity hier auf diesem Blog?

Projektziel: Mein Ziel war es, einen persönlichen Blog zu erstellen, auf dem ich meine Gedanken und Erfahrungen rund um Webentwicklung, Technologie und allem was dazugehört teilen kann.

Die Seite sollte schnell, schnörkellos und vor allem **maximal unter meiner Kontrolle** sein. Ich wollte verstehen, wie jeder Teil funktioniert, vom Backend bis zum Frontend.

Mein Setup:

1. **Backend / Content Management:** Hier kommt [Sanity.io](https://sanity.io) ins Spiel. Ich nutze den gehosteten Service von Sanity (den "Content Lake") für die Datenspeicherung und das **Sanity Studio** (ebenfalls bei Sanity gehostet) zur Pflege meiner Blogartikel, Kategorien und anderer Inhalte.
2. **Frontend / Website:** Für die eigentliche Website, die du gerade siehst, habe ich mich bewusst **gegen ein großes JavaScript-Framework** wie React, Vue oder Angular entschieden. Stattdessen setze ich auf die Grundlagen:
 - **HTML:** Für die Struktur der Seite.
 - **CSS:** Für das Styling und Layout (mit ein wenig Hilfe von SCSS für bessere Organisation).
 - **JavaScript (Vanilla JS):** Für die Dynamik – genauer gesagt, um die Inhalte von der Sanity API abzurufen und in die HTML-Struktur einzufügen.

Warum kein Framework für den Blog?

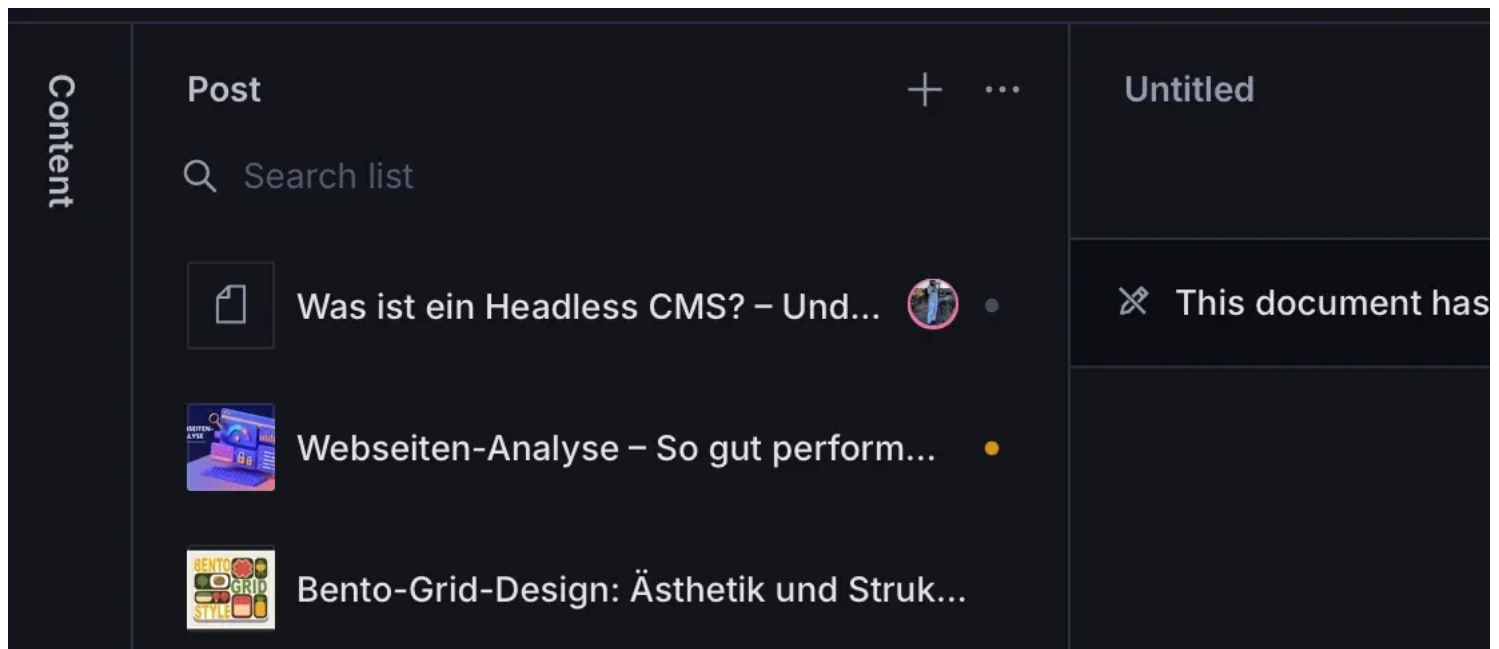
Das war eine bewusste Entscheidung aus mehreren Gründen:

- **Maximale Kontrolle & Verständnis:** Ich wollte genau nachvollziehen, was im Browser passiert. Ohne die Abstraktion eines Frameworks habe ich direkten Zugriff auf das DOM und kann die Performance bis ins Detail optimieren. Es ist auch eine gute Übung, um die Web-Grundlagen fit zu halten!
- **Demonstration der Flexibilität:** Ich wollte zeigen, dass Headless CMS nicht nur etwas für die "React/Vue-Bubble" sind. Sanity (und Headless im Allgemeinen) funktioniert wunderbar auch mit einem einfachen, klassischen Web-Setup. Man braucht nicht zwingend einen komplexen Build-Prozess oder ein schweres Framework, um die Vorteile zu nutzen.
- **Keep it simple:** Für einen relativ einfachen Blog erschien mir der Overhead eines großen Frameworks nicht notwendig. Weniger Code bedeutet oft weniger potenzielle Fehlerquellen und schnellere Ladezeiten.

Das Herzstück: Meine selbstgebaute Content-Struktur (Schema)

Das wirklich Coole an Sanity ist, dass ich die Struktur meiner Blogartikel exakt so definieren konnte, wie ich sie mir vorgestellt habe. Ich habe mir überlegt: Was braucht ein guter Blogartikel auf *meiner* Seite? Das Ergebnis ist ein Schema für den Inhaltstyp "Blog Post" mit Feldern wie:

- **title:** Der Haupttitel des Artikels (Text, erforderlich).
- **slug:** Die eindeutige URL-Endung (z. B. was-ist-ein-headless-cms), wird automatisch aus dem Titel generiert, kann aber angepasst werden (Slug-Typ, erforderlich).
- **publishedAt:** Das Veröffentlichungsdatum (Datum & Uhrzeit, erforderlich).
- **mainImage:** Das Hauptbild für den Artikel (Bild-Typ, mit Feldern für Alternativtext).
- **intro:** Eine kurze Einleitung oder Zusammenfassung (Textarea).
- **blockContent:** Der Hauptinhalt des Artikels. Hier nutze ich Sanity's **Portable Text**. Das ist super mächtig! Es ist im Grunde ein Array aus Blöcken (Absätze, Überschriften, Listen, Zitate), die man im Studio wie in einem Rich-Text-Editor bearbeiten kann. Man kann sogar eigene benutzerdefinierte Blöcke oder Inline-Elemente definieren (z. B. für Code-Snippets mit Syntax Highlighting oder spezielle Hervorhebungen).
- **keywords:** Eine Liste von relevanten Schlagwörtern für SEO und interne Verlinkung (Array aus Strings).
- **category:** Eine Verknüpfung zu einer oder mehreren Kategorien (Referenz auf den Inhaltstyp "Category").



	UI vs. UX – Was ist der Unterschied (u...
	Wie Micro-Animationen die User Exper...
	Webdesign-Typografie: 5 essenzielle ...
	Webseiten Kosten - So Viel Kostet Dei...
	Die Macht der Farben im Webdesign: ...
	Webdesign Trends 2025: Die Revolutio...
	Die Kunst des Re-Designs: Wann und ...
	SEO-Basics Part 3: Technisches SEO –...
	SEO-Basics Part 2: Off-Page SEO für ...
	SEO-Basics Part 1: On-Page-SEO für b...
	Page Speed verbessern: Web-Perform...
	Von Anfängerfehlern zur fertigen Web...
	Mehr als nur Optik: Warum Webdesign...

Dieses Schema habe ich als JavaScript-Datei definiert und in mein Sanity Studio integriert. Wenn ich jetzt einen neuen Artikel schreibe, sehe ich genau diese Felder im Redaktions-Backend. Keine überflüssigen Optionen, kein Raten, wo was hingehört – alles ist maßgeschneidert.

Wie kommen die Inhalte auf die Seite?

Wenn du eine Seite wie diese hier aufrufst, passiert (vereinfacht gesagt) Folgendes im Hintergrund:

1. Mein bisschen JavaScript im Frontend erkennt, welcher Artikel angefordert wird (anhand der URL).
2. Wenn du eine Seite wie diese hier aufrufst, kommt der Inhalt nicht live per API – sondern wurde **bereits beim Deployment abgefragt**. Mein Build-Skript (z. B. via Node oder ein kleiner Static Site Generator) stellt eine GROQ-Query an die Sanity API, holt die Inhalte als JSON und generiert daraus statisches HTML.
3. Das bedeutet: **Alle Seiten liegen schon fertig vor**, wenn der Besucher sie aufruft – schnell, sicher und unabhängig vom CMS.

Das Ergebnis ist eine statisch generierte Website, deren Inhalte aus einem flexiblen, strukturierten Backend stammen – und die im Frontend mit einfachen, robusten Web-Technologien umgesetzt ist. Für mich die perfekte Kombination aus Kontrolle, Performance und Freiheit! 🚀

5. Fazit: Sanity – Flexibel, Modern, Entwicklerfreundlich

Puh, das war ein tiefer Einblick! Fassen wir nochmal zusammen:

Ein **Headless CMS** trennt die Inhaltsverwaltung (Backend) von der Präsentation (Frontend). Das gibt dir als Entwickler maximale Freiheit bei der Wahl deiner Technologien und ermöglicht es, Inhalte über verschiedene Kanäle auszuspielen.

[Sanity.io](#) ist ein herausragendes Beispiel für ein modernes Headless CMS. Seine Stärken liegen in:

- Der **extremen Flexibilität** durch selbst definierbare Content Schemas.
- Dem **anpassbaren Sanity Studio** als Redaktions-Backend.
- Den **mächtigen APIs** (insbesondere GROQ) zum Abrufen der Daten.
- Der **hervorragenden Entwickler-Experience** und Skalierbarkeit.

Wie mein eigenes Blog-Projekt zeigt, musst du nicht zwingend auf komplexe Frameworks wie React oder Vue setzen, um Sanity zu nutzen. Es funktioniert genauso gut mit **klassischem HTML, CSS und JavaScript**, wenn du das bevorzugst oder die volle Kontrolle behalten möchtest. Natürlich glänzt Sanity auch im Zusammenspiel mit modernen Frontend-Frameworks und ermöglicht blitzschnelle Seiten durch Techniken wie Static Site Generation.

Wenn du also ein Projekt planst, bei dem du die Inhaltsstruktur genau definieren möchtest, maximale Freiheit bei der Frontend-Technologie brauchst und Wert auf einen modernen, entwicklerfreundlichen Workflow legst, dann solltest du dir Sanity definitiv genauer anschauen. Es ist ein mächtiges Werkzeug, das dir hilft, deine digitalen Ideen genau so umzusetzen, wie du sie dir vorstellst.

Ich hoffe, dieser Einblick in die Welt der Headless CMS und meine Erfahrungen mit Sanity waren hilfreich für dich. Hast du schon Erfahrungen mit Sanity oder anderen Headless-Systemen gemacht? Oder hast du Fragen dazu? Lass es mich gerne in den Kommentaren wissen (sobald ich eine Kommentarfunktion eingebaut habe 😊 – vielleicht ein nächstes Projekt?).

Bis zum nächsten Mal und viel Spaß beim Entwickeln!

Falls du dir die Arbeit nicht machen willst aber auch von einem Headless-Cms profitieren möchtest melde dich gerne bei mir, ich helfe gerne !

Weitere Artikel

[Mehr als nur Optik: Warum Webdesign entscheidend ist](#)

[04.10.2024](#)

[Von Anfängerfehlern zur fertigen Webseite: Meine Reise in der Webentwicklung](#)

[13.10.2024](#)

[Page Speed verbessern: Web-Performance-Tipps für schnellere Ladezeiten](#)

[22.10.2024](#)

[SEO-Basics Part 1: On-Page-SEO für besseres Ranking](#)

[02.11.2024](#)

[SEO-Basics Part 2: Off-Page SEO für mehr Sichtbarkeit und Vertrauen](#)

[16.11.2024](#)

Die Kunst des Re-Designs: Wann und warum deine Webseite einen neuen Look braucht

22.12.2024

Webdesign Trends 2025: Die Revolution des digitalen Designs 🚀

12.01.2025

Die Macht der Farben im Webdesign: Mehr als nur bunte Pixel 🌈

25.01.2025

Webseiten Kosten - So Viel Kostet Deine Webseite Wirklich!

08.02.2025

Webdesign-Typografie: 5 essenzielle Tipps für deine Webseite 📖

15.02.2025

UI vs. UX – Was ist der Unterschied (und warum du beide brauchst)?

09.03.2025

Bento-Grid-Design: Ästhetik und Struktur im modernen Webdesign

22.03.2025

Webseiten-Analyse – So gut performt deine Webseite wirklich!

29.03.2025

Startup-Webseite erstellen: Der komplette Guide für Gründer (inkl. Kosten & Checkliste)

12.04.2025

[zurück zum Blog](#)